Lab Sheet: 1

PART A – COLLEGE DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

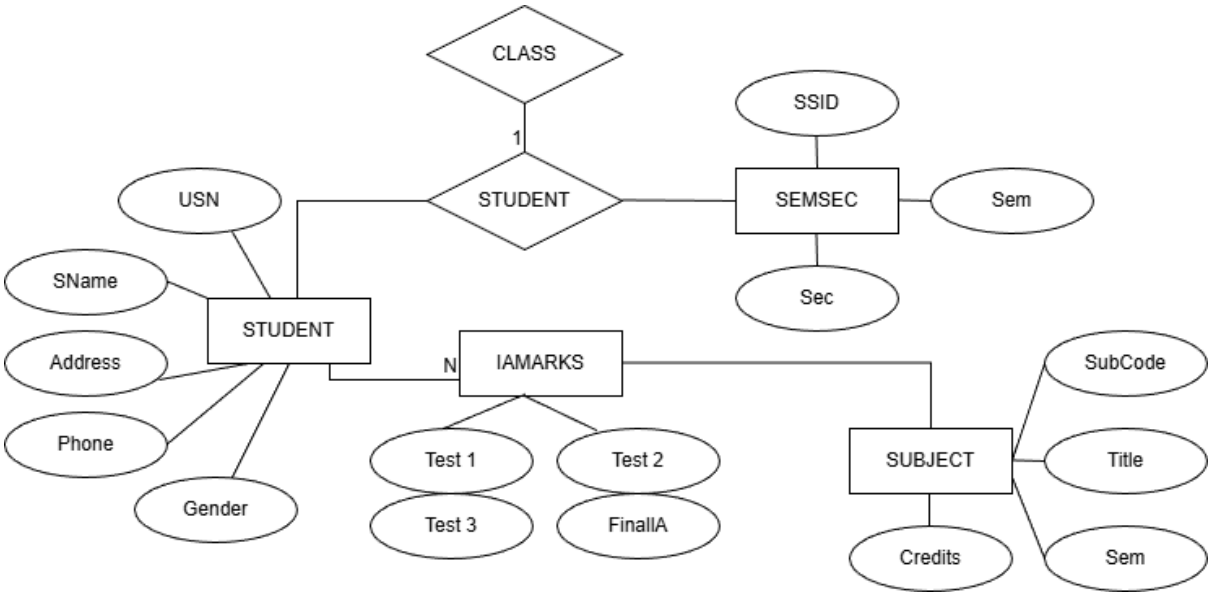
STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database

1. Introduction

The ER diagram is designed for the College Database system. It represents the structure of the database by identifying entities, their attributes, and the relationships between them. The main aim is to store and manage student details, semester-section information, subject details, and internal assessment marks.

---

2. Entities and Attributes

1. STUDENT

The STUDENT entity represents students of the college.

Attributes:

- **USN** (Primary Key)
- SName
- Address
- Phone
- Gender

USN uniquely identifies each student. No two students can have the same USN.

---

2. SEMSEC (Semester Section)

The SEMSEC entity represents a specific semester and section.

Attributes:

- **SSID** (Primary Key)
- Sem
- Sec

SSID uniquely identifies each semester-section combination.

---

3. SUBJECT

The SUBJECT entity represents subjects offered in different semesters.

Attributes:

- **Subcode** (Primary Key)
- Title
- Sem
- Credits

Subcode uniquely identifies each subject.

---

3. Relationships

1. CLASS Relationship

The CLASS relationship connects STUDENT and SEMSEC.

Meaning:

- Each student belongs to one semester and one section.

- One semester-section can have many students.

Cardinality:

- One SEMSEC → Many STUDENTS (1:M)

This relationship helps to identify which student belongs to which semester and section.

The relation table for CLASS contains:

- USN (Foreign Key)
- SSID (Foreign Key)

---

2. IAMARKS Relationship

The IAMARKS relationship connects STUDENT, SUBJECT, and SEMSEC.

Attributes:

- Test1
- Test2
- Test3
- FinalIA

Meaning:

- A student studies many subjects.
- A subject is studied by many students.
- Marks are recorded for each student in each subject.

Cardinality:

- Many-to-Many (M:N) between STUDENT and SUBJECT.

IAMARKS is considered a relationship with attributes because marks depend on both student and subject.

The relation table IAMARKS includes:

- USN (Foreign Key)
- Subcode (Foreign Key)
- SSID (Foreign Key)
- Test1
- Test2
- Test3
- FinalIA

---

4. How the ER Diagram is Constructed

1. The main entities (STUDENT, SEMSEC, SUBJECT) are identified from the given data.
2. Primary keys are selected to uniquely identify each entity.
3. Relationships are identified:
    - CLASS to connect student and semester-section.
    - IAMARKS to store internal assessment marks.
4. Cardinality is defined:
    - 1:M between SEMSEC and STUDENT.
    - M:N between STUDENT and SUBJECT.
5. Attributes are connected to their respective entities and relationships.

---

5. Conclusion

The ER diagram clearly represents the structure of the college database system. It organizes student details, semester-section details, subject details, and internal assessment marks in a structured manner. The relationships ensure proper linkage between entities and help maintain data consistency.

PART B – COMPANY DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:

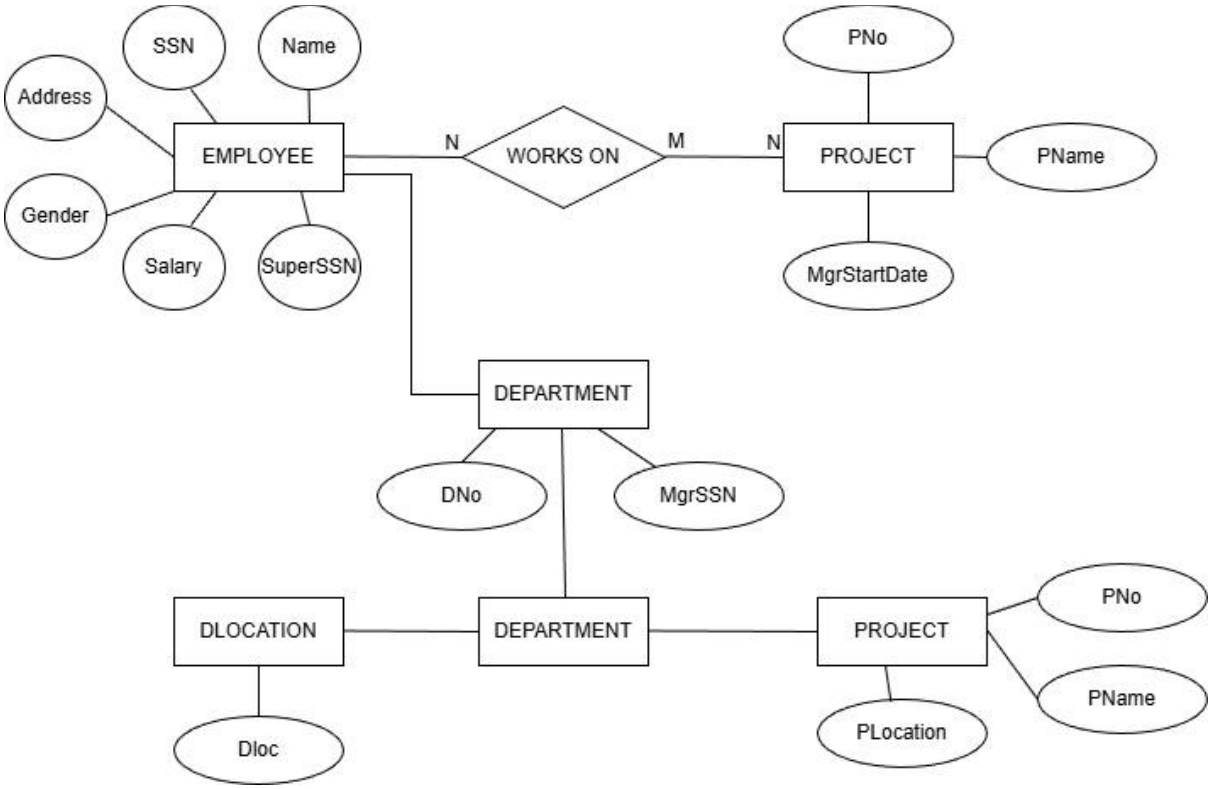
EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

DLOCATION (DNo,DLoc)

PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)



Explanation of ER Diagram – Company Database

1. Introduction

The ER diagram represents the structure of a Company Database system. It identifies the main entities such as employees, departments, projects, and locations, along with the relationships between them. The purpose of this ER diagram is to manage employee details, department information, project assignments, and work hours.

---

2. Entities and Attributes

1. EMPLOYEE

The EMPLOYEE entity represents the employees working in the company.

Attributes:

- **SSN** (Primary Key)
- Name
- Address
- Sex
- Salary
- SuperSSN
- DNo

SSN uniquely identifies each employee.
SuperSSN represents the supervisor of the employee (self-referencing relationship).
DNo indicates the department to which the employee belongs.

---

2. DEPARTMENT

The DEPARTMENT entity represents different departments in the company.

Attributes:

- **DNo** (Primary Key)
- DName
- MgrSSN
- MgrStartDate

DNo uniquely identifies each department.
MgrSSN represents the manager of the department.

---

3. PROJECT

The PROJECT entity represents projects handled by the company.

Attributes:

- **PNo** (Primary Key)
- PName
- PLocation

- DNo

PNo uniquely identifies each project.
DNo indicates which department controls the project.

---

4. DLOCATION

The DLOCATION entity represents the locations of departments.

Attributes:

- DNo
- DLoc

A department can have multiple locations.

---

3. Relationships

1. WORKS_ON Relationship

The WORKS_ON relationship connects EMPLOYEE and PROJECT.

Attribute:

- Hours

Meaning:

- An employee can work on multiple projects.
- A project can have multiple employees working on it.

Cardinality:

- Many-to-Many (M:N)

The WORKS_ON table contains:

- SSN (Foreign Key)
- PNo (Foreign Key)
- Hours

---

2. Employee–Department Relationship

- Each employee works in one department.
- One department can have many employees.

Cardinality:

- 1:M (One department → Many employees)

---

3. Department–Project Relationship

- One department controls many projects.
- Each project is controlled by one department.

Cardinality:

- 1:M (Department to Project)

---

4. Department–Location Relationship

- One department can have multiple locations.
- Each location belongs to one department.

Cardinality:

- 1:M (Department to DLOCATION)

---

4. How the ER Diagram is Constructed

1. The main entities (EMPLOYEE, DEPARTMENT, PROJECT, DLOCATION) are identified.
2. Primary keys are assigned to uniquely identify each entity.
3. Relationships are identified:
    - WORKS_ON between EMPLOYEE and PROJECT.
    - Department–Employee relationship.
    - Department–Project relationship.
    - Department–Location relationship.
4. Cardinality is defined:
    - 1:M between Department and Employee.
    - 1:M between Department and Project.
    - 1:M between Department and Location.
    - M:N between Employee and Project.
5. Relationship attributes such as Hours are added to WORKS_ON.

5. Conclusion

The ER diagram clearly represents the structure of the company database. It shows how employees are assigned to departments, how departments manage projects, and how employees work on multiple projects. The relationships ensure proper organization and efficient management of company data.

PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with appropriate integrity constraints

```
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.35 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CustomerSaleDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE CustomerSaleDB;
Database changed
mysql> CREATE TABLE Customer (
    ->     cust_id INT PRIMARY KEY,
    ->     cust_name VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Item (
    ->     item_id INT PRIMARY KEY,
    ->     item_name VARCHAR(100) NOT NULL,
    ->     price INT NOT NULL CHECK (price > 0)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Sale (
    ->     bill_no INT PRIMARY KEY,
    ->     bill_date DATE NOT NULL,
    ->     cust_id INT,
    ->     item_id INT,
    ->     qty_sold INT CHECK (qty_sold > 0),
    ->     FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    ->     FOREIGN KEY (item_id) REFERENCES Item(item_id)
    -> );
Query OK, 0 rows affected (0.09 sec)
```

2. Insert around 10 records in each table

```
mysql> INSERT INTO Customer VALUES
    -> (1,'Aman'),
    -> (2,'Riya'),
    -> (3,'Karan'),
    -> (4,'Sneha'),
    -> (5,'Rahul'),
    -> (6,'Megha'),
    -> (7,'Arjun'),
    -> (8,'Pooja'),
    -> (9,'Vikas'),
    -> (10,'Neha');
Query OK, 10 rows affected (0.03 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Item VALUES
    -> (101,'Pen',20),
    -> (102,'Notebook',80),
    -> (103,'Bag',500),
    -> (104,'Shoes',1200),
    -> (105,'Bottle',250),
    -> (106,'Watch',1500),
    -> (107,'Mobile Cover',300),
    -> (108,'Headphones',2000),
    -> (109,'Keyboard',1800),
    -> (110,'Mouse',400);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Sale VALUES
    -> (1,CURDATE(),1,103,2),
    -> (2,CURDATE(),5,104,1),
    -> (3,CURDATE(),2,101,5),
    -> (4,'2026-02-15',3,105,2),
    -> (5,'2026-02-16',5,108,1),
    -> (6,CURDATE(),6,110,3),
    -> (7,'2026-02-14',4,102,4),
    -> (8,CURDATE(),5,106,1),
    -> (9,'2026-02-12',7,109,1),
    -> (10,CURDATE(),8,107,2);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

3. List all the bills for the current date with customer names and item numbers

```
mysql> SELECT s.bill_no, c.cust_name, s.item_id
    -> FROM Sale s
    -> JOIN Customer c ON s.cust_id = c.cust_id
    -> WHERE s.bill_date = CURDATE();
+---------+-----------+---------+
| bill_no | cust_name | item_id |
+---------+-----------+---------+
|       1 | Aman      |     103 |
|       2 | Rahul     |     104 |
|       3 | Riya      |     101 |
|       6 | Megha     |     110 |
|       8 | Rahul     |     106 |
|      10 | Pooja     |     107 |
+---------+-----------+---------+
6 rows in set (0.02 sec)
```

4. List the total Bill details with quantity sold, price of item and final amount

```
mysql> SELECT s.bill_no,
    ->        s.qty_sold,
    ->        i.price,
    ->        (s.qty_sold * i.price) AS total_amount
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id;
+---------+----------+-------+--------------+
| bill_no | qty_sold | price | total_amount |
+---------+----------+-------+--------------+
|       1 |        2 |   500 |         1000 |
|       2 |        1 |  1200 |         1200 |
|       3 |        5 |    20 |          100 |
|       4 |        2 |   250 |          500 |
|       5 |        1 |  2000 |         2000 |
|       6 |        3 |   400 |         1200 |
|       7 |        4 |    80 |          320 |
|       8 |        1 |  1500 |         1500 |
|       9 |        1 |  1800 |         1800 |
|      10 |        2 |   300 |          600 |
+---------+----------+-------+--------------+
10 rows in set (0.01 sec)
```

5. List the details of customers who have bought a product with price > 200

```
mysql> SELECT DISTINCT c.cust_id, c.cust_name
    -> FROM Customer c
    -> JOIN Sale s ON c.cust_id = s.cust_id
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE i.price > 200;
+---------+-----------+
| cust_id | cust_name |
+---------+-----------+
|       1 | Aman      |
|       5 | Rahul     |
|       3 | Karan     |
|       8 | Pooja     |
|       7 | Arjun     |
|       6 | Megha     |
+---------+-----------+
6 rows in set (0.01 sec)
```

6. Give count of how many products have been bought by each customer

```
mysql> SELECT c.cust_name,
    ->        COUNT(s.item_id) AS total_products_bought
    -> FROM Customer c
    -> LEFT JOIN Sale s ON c.cust_id = s.cust_id
    -> GROUP BY c.cust_name;
+-----------+----------------------+
| cust_name | total_products_bought |
+-----------+----------------------+
| Aman      |                    1 |
| Riya      |                    1 |
| Karan     |                    1 |
| Sneha     |                    1 |
| Rahul     |                    3 |
| Megha     |                    1 |
| Arjun     |                    1 |
| Pooja     |                    1 |
| Vikas     |                    0 |
| Neha      |                    0 |
+-----------+----------------------+
10 rows in set (0.01 sec)
```

7. Give list of products bought by customer having cust_id = 5

```
mysql> SELECT i.item_name
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE s.cust_id = 5;
+------------+
| item_name  |
+------------+
| Shoes      |
| Headphones |
| Watch      |
+------------+
3 rows in set (0.01 sec)
```

8. List the item details which are sold as of today

```
mysql> SELECT i.item_id, i.item_name, i.price
    -> FROM Item i
    -> JOIN Sale s ON i.item_id = s.item_id
    -> WHERE s.bill_date = CURDATE();
+---------+--------------+-------+
| item_id | item_name    | price |
+---------+--------------+-------+
|     103 | Bag          |   500 |
|     104 | Shoes        |  1200 |
|     101 | Pen          |    20 |
|     110 | Mouse        |   400 |
|     106 | Watch        |  1500 |
|     107 | Mobile Cover |   300 |
+---------+--------------+-------+
6 rows in set (0.00 sec)
```

9. Create a view listing bill_no, bill_date, cust_id, item_id, price, qty_sold, amount

```
mysql> CREATE VIEW Bill_Details AS
    -> SELECT s.bill_no,
    ->        s.bill_date,
    ->        s.cust_id,
    ->        s.item_id,
    ->        i.price,
    ->        s.qty_sold,
    ->        (s.qty_sold * i.price) AS amount
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Bill_Details;
+---------+------------+---------+---------+-------+----------+--------+
| bill_no | bill_date  | cust_id | item_id | price | qty_sold | amount |
+---------+------------+---------+---------+-------+----------+--------+
|       1 | 2026-02-18 |       1 |     103 |   500 |        2 |   1000 |
|       2 | 2026-02-18 |       5 |     104 |  1200 |        1 |   1200 |
|       3 | 2026-02-18 |       2 |     101 |    20 |        5 |    100 |
|       4 | 2026-02-15 |       3 |     105 |   250 |        2 |    500 |
|       5 | 2026-02-16 |       5 |     108 |  2000 |        1 |   2000 |
|       6 | 2026-02-18 |       6 |     110 |   400 |        3 |   1200 |
|       7 | 2026-02-14 |       4 |     102 |    80 |        4 |    320 |
|       8 | 2026-02-18 |       5 |     106 |  1500 |        1 |   1500 |
|       9 | 2026-02-12 |       7 |     109 |  1800 |        1 |   1800 |
|      10 | 2026-02-18 |       8 |     107 |   300 |        2 |    600 |
+---------+------------+---------+---------+-------+----------+--------+
10 rows in set (0.00 sec)
```

10. Create a view for daily sales date-wise for last one week

```
mysql> CREATE VIEW Weekly_Sales AS
    -> SELECT s.bill_date,
    ->        SUM(s.qty_sold * i.price) AS total_sales
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
    -> GROUP BY s.bill_date;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Weekly_Sales;
+------------+-------------+
| bill_date  | total_sales |
+------------+-------------+
| 2026-02-18 |        5600 |
| 2026-02-15 |         500 |
| 2026-02-16 |        2000 |
| 2026-02-14 |         320 |
| 2026-02-12 |        1800 |
+------------+-------------+
5 rows in set (0.01 sec)
```